LAPORAN UTS IMPLEMENTASI
SISTEM INVENTARIS TOKO / GUDANG
SE-2

DOSEN PENGAMPU: Muhammad Panji Muslim, S.Pd., M.Kom

ANDITO DWI WICAKSONO

2410511115

FAKULTAS ILMU KOMPUTER

PROGRAM STUDI INFORMATIKA

UPN VETERAN JAKARTA

TAHUN 2026

DAFTAR ISI

BAB I PENDAHULUAN 3

1.1 Arsitektur Sistem 3

1.2 Justifikasi Microservices dibanding Monolitik 3

BAB II IMPLEMENTASI REST API & DATABASE 5

2.1 Auth Service 5

2.2 Inventory Service 6

2.3 Log Service 6

BAB III KEAMANAN & *INTERSERVICE COMMUNICATION* 8

3.1 JSON Web Token (JWT) 8

3.2 GitHub Oauth 8

3.3 Routing Map & Komunikasi Antar-Service 8

BAB IV HASIL PENGUJIAN 9

4.1 Pengujian dengan Postman 9

BAB V KESIMPULAN 10

LAMPIRAN SCREENSHOT 11

BAB I
PENDAHULUAN

1.1 Arsitektur Sistem

Sistem Inventaris ini dibangun menggunakan arsitektur *microservices* dengan MySQL sebagai sistem manajemen database utama di seluruh layanan. Arsitektur ini menggunakan 3 service independen dan 1 API Gateway sebagai gerbang utama:

a. API Gateway (Node.js) = Bertugas sebagai *single entry point* (Port 3000), menangani routing, dan melakukan *path rewrite* untuk menyembunyikan struktur internal port service.
b. Auth-Service (Node.js & MySQL) = Mengelola registrasi pengguna, login manual (JWT), dan integrasi GitHub OAuth 2.0.
c. Inventory-Service (CodeIgniter 4 & MySQL) = Mengelola manajemen data barang dan kategori stok.
d. Log-Service (Node.js & MySQL) = Mencatat setiap aktivitas perubahan data yang dilakukan melalui Gateway ke dalam tabel audit.

1.2 Justifikasi Microservices dibanding Monolitik

Implementasi arsitektur *microservices* pada proyek ini didasarkan pada kebutuhan akan fleksibilitas dan ketangguhan sistem yang tidak dapat dicapai secara optimal oleh arsitektur monolitik. Beberapa alasan utama pemilihan arsitektur ini adalah:

a. *Fault Isolation* (Isolasi Kesalahan) = Pemisahan layanan memastikan bahwa kegagalan pada satu *service* tidak akan melumpuhkan seluruh sistem. Sebagai contoh, apabila Log-Service mengalami gangguan akibat lonjakan trafik pencatatan aktivitas, fungsi utama pada Inventory-Service seperti pengecekan stok barang tetap dapat beroperasi secara normal tanpa interupsi.
b. *Independent Scalability* (Skalabilitas Independen) = Setiap layanan dapat ditingkatkan kapasitasnya secara terpisah sesuai dengan beban kerja masing-

masing. Auth-Service yang menangani proses autentikasi (termasuk GitHub OAuth) dapat dikembangkan secara mandiri untuk menangani lonjakan beban login tanpa harus mereplikasi seluruh komponen sistem inventaris lainnya.

c. *Deployment Independence* = Pengembangan fitur baru pada satu layanan dapat dilakukan tanpa harus melakukan *re-deploy* pada seluruh aplikasi. Hal ini sangat efektif mengingat Inventory-Service menggunakan framework CodeIgniter 4 sementara layanan pendukung lainnya berbasis Node.js, sehingga pembaruan pada salah satu *stack* teknologi tidak mengganggu stabilitas layanan lainnya.

d. *Data Sovereignty* (Kedaulatan Data) = Dengan prinsip *database-per-service*, setiap layanan mengelola skema datanya sendiri dalam database MySQL yang terpisah (misalnya db_logs dan db_inventaris). Hal ini mencegah adanya ketergantungan tabel (*direct join*) antar layanan yang sering kali menjadi masalah pada arsitektur monolitik berskala besar.

BAB II
IMPLEMENTASI REST API & DATABASE

2.1 Auth Service

Service ini bertanggung jawab penuh atas manajemen identitas pengguna. Penggunaan MySQL pada layanan ini menjamin integritas data pengguna yang terstruktur dan mempermudah sinkronisasi profil antara akun lokal dan akun pihak ketiga (GitHub).

a. *Schema Database* = Menggunakan tabel users dengan kolom utama id, email (*unique*), password (*hashed* menggunakan bcrypt), name, github_id (untuk integrasi OAuth), serta avatar_url.
b. Endpoint Utama:
    - POST /auth/register = Melakukan validasi duplikasi email dan melakukan *hashing* pada password sebelum disimpan ke database.
    - POST /auth/login = Memvalidasi kredensial pengguna dan menghasilkan sepasang JSON Web Token (JWT) berupa *Access Token* dan *Refresh Token*.
    - GET /auth/github/callback = Menangani proses pertukaran kode dari GitHub menjadi profil pengguna, serta melakukan pemetaan otomatis ke *database* lokal jika pengguna baru pertama kali melakukan login.
c. Validasi dengan setiap *input* diproses melalui *middleware* validasi di tingkat *controller* untuk memastikan data yang masuk ke *database* MySQL memenuhi standar keamanan.

Alasan menggunakan MySQL di Auth Service adalah skema tabel harus didefinisikan secara kaku sejak awal. Jika ada tambahan *provider* OAuth lain di masa depan yang memiliki atribut profil berbeda, maka harus melakukan migrasi skema (*Alter Table*).

2.2 Inventory Service

Layanan ini mengelola data inti inventaris dengan arsitektur MVC yang disediakan oleh *framework* CodeIgniter 4. MySQL dipilih karena kemampuannya menangani relasi kompleks antar data barang.

a. HTTP Semantics:
   - 200 OK =Digunakan untuk pengambilan daftar barang (list).
   - 201 Created = Digunakan untuk konfirmasi keberhasilan penambahan barang atau kategori baru.
   - 404 Not Found = Jika data barang yang dicari berdasarkan ID tidak ditemukan dalam *database*.

b. Endpoint Utama:
   - GET /inventory = Menampilkan seluruh daftar barang yang tersedia. Implementasi ini mencakup fitur *Paging & Filtering* (default 5 item per halaman) untuk mengoptimalkan performa.
   - POST /inventory/add = Digunakan untuk menambah stok atau item baru ke dalam sistem.

Pembagian *database* dilakukan dengan setiap service memiliki *database schema* sendiri (misalnya db_logs, db_inventaris). Hal ini menjaga prinsip *Database-per-Service* agar tidak ada ketergantungan langsung antar tabel di tingkat *database*, sehingga servis tetap bersifat *loosely coupled*.

2.3 Log Service

Service ini berperan sebagai sistem audit yang bersifat *data-driven* untuk memantau setiap aktivitas perubahan data dalam sistem inventaris.

a. *Schema Database* = Tabel activity_logs menyimpan detail aksi seperti user_id, action (Create/Update/Delete), impacted_item_id, dan timestamp.

b. Fitur Utama (Audit Trail):
   - Automated Logging = Setiap kali terjadi transaksi atau perubahan data barang di *Inventory Service*, sistem akan mengirimkan data log ke

layanan ini melalui API Gateway.

- Integritas Data = Penyimpanan di MySQL memastikan bahwa histori aktivitas tidak mudah dimanipulasi dan dapat digunakan untuk keperluan audit di masa mendatang.

c. Endpoint Utama:

- GET /logs = Menampilkan riwayat aktivitas pengguna untuk keperluan pengawasan sistem.

Alasan memilih MySQL untuk semua service, yakni untuk memastikan standar keamanan data yang seragam. MySQL memungkinkan untuk melakukan *query* yang kompleks (Join) antara data pengguna dan data aktivitas secara lebih efisien jika nantinya dibutuhkan pelaporan terpusat.

BAB III
KEAMANAN & *INTERSERVICE COMMUNICATION*

3.1 JSON Web Token (JWT)

Sistem ini mengimplementasikan JSON Web Token sebagai metode autentikasi utama untuk mengamankan akses antar-layanan. JWT digunakan untuk memastikan bahwa setiap permintaan yang masuk ke *Inventory-Service* dan *Log-Service* berasal dari pengguna yang terverifikasi.

a. Login = Setelah verifikasi berhasil, sistem menghasilkan access_token (masa berlaku 15 menit) dan refresh_token (masa berlaku 7 hari).
b. Middleware = Setiap rute terproteksi pada *Inventory-Service* menggunakan authMiddleware untuk memvalidasi token JWT di setiap *request* sebelum data diproses.

3.2 GitHub Oauth

Sistem mengintegrasikan GitHub OAuth 2.0 menggunakan *Authorization Code Flow*. Data pengguna seperti nama, email, dan URL avatar diambil melalui API GitHub dan disimpan ke dalam database MySQL lokal dengan penanda provider: "github" untuk memfasilitasi login tanpa *password* di masa mendatang.

3.3 Routing Map & Komunikasi Antar-Service

API Gateway berjalan pada port 3000 dan mengelola lalu lintas permintaan sebagai berikut:

a. /auth/ = http://localhost:3001 (Auth-Service)
b. /inventory/ = http://localhost:8080 (Inventory-Service)
c. /logs/ = http://localhost:3003 (Log-Service)

Komunikasi antar-layanan dilakukan secara asinkron melalui Gateway dengan setiap aksi manipulasi data pada *Inventory-Service* akan memicu pengiriman data log ke Log-Service melalui *HTTP Request* menggunakan library Axios.

BAB IV
HASIL PENGUJIAN

4.1 Pengujian dengan Postman

Pengujian dilakukan melalui API Gateway (Port 3000) sebagai *single entry point* untuk memverifikasi fungsionalitas seluruh layanan.

- Otentikasi = Verifikasi registrasi user, login lokal untuk mendapatkan JWT, serta pengujian rute GitHub OAuth untuk memastikan pemetaan profil pengguna ke database MySQL berjalan lancar.
- Manajemen Inventaris = Pengujian operasi CRUD pada *Inventory-Service* (CI 4) melalui Gateway, termasuk pengecekan parameter paginasi dan filter barang.
- Aktivitas Logging = Memastikan bahwa setiap perubahan data pada inventaris berhasil tercatat secara otomatis di Log-Service dan tersimpan dalam database db_logging.
- Middleware Keamanan = Memastikan akses tanpa token yang valid atau menggunakan token kadaluwarsa akan menghasilkan status 401 *Unauthorized*.

BAB V
KESIMPULAN

Arsitektur *microservices* yang diimplementasikan mulai dari pemisahan *database* antar-layanan hingga integrasi layanan pihak ketiga (GitHub OAuth). Penggunaan PHP MVC (CodeIgniter 4) pada *Inventory-Service* memberikan struktur manajemen data yang solid, sementara penggunaan Node.js pada Gateway, Auth, dan Log service memastikan performa komunikasi antar-layanan yang efisien dan responsif.

LAMPIRAN SCREENSHOT

POST http://localhost:3000/auth/register

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "email": "anditodwi03@gmail.com",
3   "password": "123"
4 }
```

Body Cookies (1) Headers (11) Test Results 201 Created 322

{} JSON Preview Visualize

```
1 {
2     "id": 1,
3     "email": "anditodwi03@gmail.com",
4     "password": "$2b$10$qJDh4V70v00t8VV6/TLP3uRDviR.sGRA/7jeQPS.FxsBDvIoFuNXi",
5     "role": "user"
```

uts_2410511115 / **Login email**

POST http://localhost:3000/auth/login

Docs Params Authorization Headers (9) Body

none form-data x-www-form-urlencoded ra

```
1 {
2     "email": "anditodwi03@gmail.com",
3     "password": "123"
4 }
```

Body Cookies (1) Headers (10) Test Results

{} JSON Preview Visualize

```
1
2 eIsInJvbGUiOiJ1c2VyIjwiaWF0IjoxNzc3NzIyMDY1LC
```

POST http://localhost:3000/inventory/barang

Docs Params Authorization Headers (10) Body

none form-data x-www-form-urlencoded raw

```
1 {
2     "nama_barang": "Laptop Asus",
3     "stok": 12,
4     "harga": 200000,
5     "id_kategori": 1,
```

Body Cookies (1) Headers (10) Test Results

{} JSON Preview Visualize

```
1 {
2     "message": "Barang berhasil ditambahkan"
3 }
```

GET http://localhost:3000/inventory/barang

Docs Params Authorization Headers (9) Body Scripts

Auth Type

Be...

Token

The authorization header will be automatically generated when you send the request. Learn more about

Body Cookies (1) Headers (10) Test Results

JSON Preview Visualize


```
"id": "2",
"nama_barang": "Keybaord Asus",
"stok": "12",
"harga": "200000.00",
"id_kategori": "1",
"id_supplier": "1",
"created_at": "2026-05-02 08:15:56",
"updated_at": "2026-05-02 08:15:56"
```


GET http://localhost:3000/inventory/barang/6

Docs Params Authorization Headers (7) Body Scripts

Query Params

Key	Value
Key	Value

Body Cookies (1) Headers (10) Test Results

JSON Preview Visualize


```
{
    "id": "6",
    "nama_barang": "Laptop Asus",
    "stok": "12",
    "harga": "200000.00",
```


DELETE http://localhost:3000/inventory/barang/7

Docs Params Authorization Headers (9) Body

uth Type

Be...

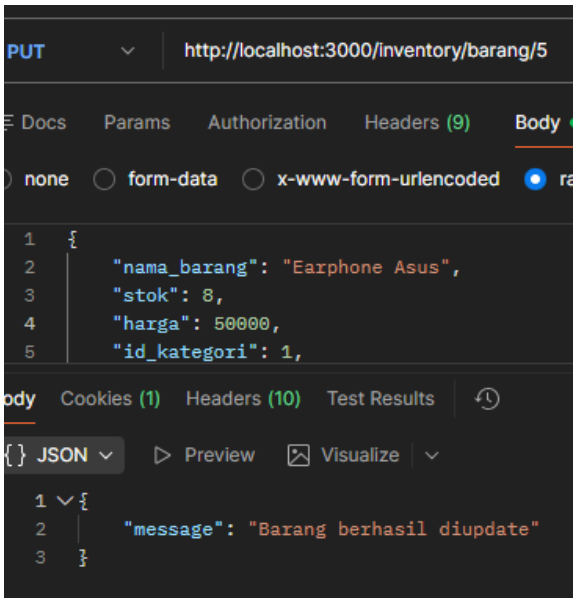
Token

he authorization header will be tomatically generated when you nd the request. Learn more about

ody Cookies (1) Headers (10) Test Results

JSON Preview Visualize


```
{
    "message": "Barang berhasil dihapus"
}
```


PUT
http://localhost:3000/inventory/barang/5
Docs
Params
Authorization
Headers (9)
Body
none
form-data
x-www-form-urlencoded
{
"nama_barang": "Earphone Asus",
"stok": 8,
"harga": 50000,
"id_kategori": 1,
Cookies (1)
Headers (10)
Test Results
JSON
Preview
Visualize
{
"message": "Barang berhasil diupdate"
}